# Hierarchical IPTC Topic Classification of French Press Articles with EuroBERT and Hierarchy-Guided Contrastive Learning

Zehua Huang

Internship report
Abstrakt, July to October 2026

# Acknowledgements

# Abstract

# Contents

**Acknowledgements** i

**Abstract** ii

**1 Introduction** 1
- 1.1 Abstrakt and AI-assisted press monitoring . . . 1
- 1.2 The mission: a topic tag on each article . . . 1
- 1.3 Objectives and constraints . . . 2
- 1.4 Contributions of the internship . . . 2
- 1.5 Outline of the report . . . 3

**2 Background** 4
- 2.1 The IPTC Media Topics tree . . . 4
- 2.2 Hierarchical multi-label classification . . . 4
- 2.3 Encoders for French press . . . 5
- 2.4 Coherent predictions: C-HMCNN and the max rule . . . 5
- 2.5 Losses for imbalanced multi-label data . . . 5
- 2.6 Evaluation . . . 6

**3 Data: 3 data sets** 8
- 3.1 Starting point . . . 8
- 3.2 The easy data set . . . 8
- 3.3 The gold training set . . . 9
- 3.4 The real test set . . . 9
- 3.5 Purpose and limits . . . 9

**4 Method** 10
- 4.1 Model overview . . . 10
- 4.2 Baselines . . . 10
  - 4.2.1 Frozen encoder and an MLP head: choosing the encoder . . . 12
  - 4.2.2 Training the encoder: warm-up, learning rates, top layers, LoRA . . . 13
  - 4.2.3 Graphormer label attention . . . 14
  - 4.2.4 Several seeds: the spread and the ensemble . . . 14
- 4.3 The model block by block . . . 15
  - 4.3.1 Topic features and the graph encoder . . . 15
  - 4.3.2 Positive-sample generator and the 2 views . . . 16
  - 4.3.3 Article-to-topic attention and token FFN . . . 16
  - 4.3.4 Attention pooling . . . 17
  - 4.3.5 Head . . . 17
  - 4.3.6 Classification loss . . . 17
  - 4.3.7 Contrastive loss and the total loss . . . 18
- 4.4 Coherence rules . . . 18

4.4.1 Rules at inference . . . 18
4.4.2 Rules in training . . . 19
4.4.3 The avg-max rule and its implementation . . . 20
4.5 Training settings . . . 20

**5 Experimental setup** **22**

**6 Results** **23**
6.1 Baselines on the easy data set: encoders, frozen and fine-tuned . . . 23
6.2 Losses and coherence rules on the easy data set . . . 23
6.3 HGCLR and our changes on the easy data set . . . 24
6.4 Summary on the easy data set . . . 25
6.5 Runs on the gold training set . . . 25
6.6 Choice of the coherence rule . . . 26
6.7 Seeds and the ensemble . . . 27

**7 Error analysis** **28**
7.1 The errors by level . . . 28
7.2 Errors on the validation data: the model or the LLM topics? . . . 28
7.3 Errors on the real test set: a different mix of topics . . . 29
7.4 Lessons for the next model . . . 31

**8 Limits** **32**

**9 Side work: relevance rerankers for thematic search** **33**
9.1 Problem and data . . . 33
9.2 zerank cross-encoders run locally . . . 33
9.3 GLiNER2 as a zero-shot classifier . . . 33
9.4 Findings . . . 33

**10 Conclusion** **34**

**A Full architecture graph** **36**

**B Configuration file and flag semantics** **37**

**C Run index** **38**

**D LLM labelling prompts** **39**

**E Experiment journal excerpts** **40**

# List of Figures

2.1 The idea of ASL, 2 figures from Ridnik et al. [5]. Top: the loss on a negative against its score, for cross-entropy, focal loss, and focal loss with the margin $m$. Bottom: the mean confidence in the right answer, $p_t$, of the positives and of the negatives during training on MS-COCO, with cross-entropy, focal loss and ASL. 6

4.1 The model: the data flow from the inputs on the top row to the topic tags. White: inputs. Blue: HGCLR blocks. Green: our changes. Green with an orange border: avg-max, our variant of the C-HMCNN rule. Solid: the inference path, one pass per article. Dashed: training only. The letters on the arrows are the symbols of the equations in Section 4.3; the positive view $\hat{e}$ follows the same chain as the article and gives $\hat{z}$ and $\hat{s}$. The losses are in Section 4.3; the full graph with tensor shapes is in Appendix A. . . . . 11

4.2 The frozen and the fine-tuned baselines share one chain. Frozen: the encoder runs once and its token vectors are stored. Fine-tune: the encoder trains with the head. White: input. Yellow: the blocks of the baselines. . . . . 13

4.3 The Graphormer label-attention baseline. The topic side on the left and the article on the right meet in the label attention, where each topic reads the article for itself. White: inputs. Yellow: the blocks of this baseline. . . . . 14

7.1 The per-topic view of the dashboard, final model, validation data, over the 291 scored topics. Left: AP of each topic against its number of training articles, log scale, 1 dot per topic, coloured by level. Right: the topics ranked by AP inside each level, with the macro AP line. . . . . 29

# List of Tables


1.1 Targets and constraints of the model. Quality is measured against the LLM topics, so the LLM topics set a ceiling for the model. . . . . 2

2.1 The 2 encoders compared in this report. Sizes in parameters; input length in tokens. 5

3.1 The 3 data sets. The easy data set is mine; the gold training set and the real test set are the team's. . . . . 8

4.1 The blocks of the model in the order of Figure 4.1, where each one comes from, and what it does. . . . . 12
4.2 The baselines, in the order they were tried on the easy data set. All share the split, the topic set, the loss family and the metrics of the HGCLR runs. . . . . 12
4.3 Settings of the blocks in the run configuration. The HGCLR rows follow the paper and its reference code; the other rows are our blocks. . . . . 15
4.4 The 5 coherence settings. The rule in training sits inside the classification loss; the rule at inference runs on $p = \sigma(s)$ before the threshold. . . . . 19
4.5 The 2 rules on one chain root $\rightarrow$ A $\rightarrow$ a. The avg-max rule averages (A, a), then (root, a), then (root, A). . . . . 19
4.6 Training settings of the final model. . . . . 21

5.1 The training environment. Table 4.6 gives the training settings. . . . . 22

6.1 **The 4 groups of runs.** Scores compare within a group, not across groups. Groups 3 and 4 share their validation split. . . . . 23
6.2 **Group 1: easy data set, old platform.** Baselines, validation split. One seed, or the ensemble of several seeds. Weighted BCE and no coherence rule unless stated. . . . . 24
6.3 **Group 1: easy data set, old platform.** The ASL sweep on the frozen baseline, 1 seed. The weighted BCE row comes from a later run of the same baseline. Macro AP is the sort key of Chapter 6. . . . . 24
6.4 **Group 1: easy data set, old platform.** Coherence rules. Each block differs in the rule alone. The last 3 columns give macro F1 with no rule, with the max rule and with the avg-max rule applied to the saved scores; a run trained with a rule in the loss has the same value in all 3. . . . . 25
6.5 **Group 1: easy data set, old platform.** HGCLR, 1 seed each, weighted BCE unless stated. Frozen rows: the positive view comes from a mask that keeps the stated share of tokens; a guided mask keeps the tokens closest to the names of the article's topics. Fine-tune rows: the positive view comes from the graph-guided generator of Section 4.3.2. The positive views of a frozen run are encoded once and cached; the 2 rows with no mask differ only in how often the cache is rebuilt, which with no mask changes the random stream alone. The plain fine-tune row repeats Table 6.2. . . . . 26

6.6 **Groups 3 and 4: pruned gold set, rebuilt pipeline**, full corpus. Compare rows within a group only. One seed unless stated. . . . . 26
6.7 The evidence for the avg-max rule. **Each pair in panel B sits inside one group of Table 6.1.** A: each rule applied after training to the saved scores of the 58 runs of the old platform trained without a rule. B: the rule inside the loss, macro AP on the validation data; a dash marks a pair that was not run. *The 2 runs also differ in the soft values and the article-to-topic attention. . . . . 27
6.8 The final design with 3 seeds and their ensemble. **Group 4**, full corpus. . . . . 27

7.1 The 32 topics the model learns and the metrics skip, by the reason found for each. 28
7.2 The final model by level. Macro AP, and precision and recall over all article-topic pairs at the threshold of 0.5. . . . . 29
7.3 4 causes of a disagreement on the validation data, 1 article each. The LLM topics and the model's topics at level 1 and 2, where they differ. . . . . 30
7.4 A validation article where the model and the LLM topics disagree. *Une retraitée d'Ajaccio tricote une écharpe de 21 kilomètres, chantier textile inédit en Corse*: a 72-year-old retired woman finishes a scarf 21 km long; the body stops at a paywall after 2 sentences. Votes: LLMs out of 3 that gave the topic; 0 means no LLM did. 30
7.5 Blind LLM judge on validation disagreements, earlier run of the same design. One judge, no human check. . . . . 30
7.6 Share of articles that carry a level 1 topic, in the generated validation data and in the real test set, and the model's results on the real test set at the threshold of 0.5. Topics with the largest change. . . . . 31
7.7 A real test page with many errors. *Service Photo : ses derniers articles publiés*, an author page of *Libération* that lists headlines: the war in the Middle East, the PSG victory, a heat wave. Missed: a topic the LLMs gave and the model scored below 0.5. Added: the reverse. . . . . 31

# Chapter 1

# Introduction

## 1.1 Abstrakt and AI-assisted press monitoring

Abstrakt is a French start-up. It sells a press monitoring service helped by AI. A customer wants to know what the press writes about a company or a public issue. Each day the service picks the articles that matter for that customer and sends them. The press publishes more articles than a person can read, so the AI reads first and a person reads what the AI kept. Abstrakt states the rule in one line: AI narrows the reading, people decide. Each automatic decision gets 1 of 3 answers, yes, no, or not sure, and a person handles the not sure cases. For a model this means 2 things. Recall comes first, because a customer does not see a missed article. Precision has a floor, because a person pays for each wrong article with reading time.

I did this internship from the end of July to October 2026 as a machine-learning intern in the engineering team. I worked on the step that gives each article its topics.

## 1.2 The mission: a topic tag on each article

Each article in the service carries topic tags. The tags come from the IPTC Media Topics list, the standard of the news industry. The topics form a tree. Level 1 has 17 broad topics, for example *politics and government* or *sport*. Level 2 has 120 topics and level 3 has 515. The project stops at level 3. Many articles have several topics. A topic implies its parent topics up to the root.

The product uses the tags in 2 places. With a level 3 tag on each article, a customer can filter the articles on one precise topic, as in a traditional press review. The level 2 tags serve as filters in the search bar.

A large language model (LLM) can read an article and classify its topics from the list. The team started this way, and the cost was the problem: **classifying each article with an LLM costs too much** at the scale of the service. My mission was to find a better way. A small model is trained once, then runs on 1 GPU and tags each article at a very low cost. A small model learns from examples, and the examples were missing. So the LLMs keep one job, writing the training data. Several LLMs vote on the topics of each training article, and the model learns from the vote. A vote of several LLMs is more reliable than 1 LLM, so the model can tag better than the single LLM the team started with.

3 more facts guided the design. Copyright forbids training the model on real articles. The LLM therefore writes training articles for each topic, and the team keeps real articles for the test data. The tree has 652 topics down to level 3. The team cut the niche topics and the topics with no business use, and of the topics that remain, many appear in a few articles. These rare topics drive the macro scores. The model runs on one small GPU in production. One encoder with a small head on top fits this GPU, so I kept that design and did not try one model per node of the tree.

The team chose the design at the start: a pre-trained text encoder, EuroBERT, and a head on top that outputs one score per topic. My work during the internship was to train the best model I could within this design. In practice this meant **choosing the encoder and its size, the way to train it, the pooling and the head, the loss, and a rule that keeps the scores coherent along the tree**; the one method that teaches the encoder the tree itself, **HGCLR, had to be rebuilt for EuroBERT**. Chapter 4 presents these choices and Chapter 6 compares them.

## 1.3 Objectives and constraints

Table 1.1 lists the targets and the constraints of the model. **The quality targets apply per level and loosen with depth**, since the LLM topics are less reliable there. **The final model meets them on the validation data**, Table 7.2. **The real test set scores lower, since its LLM topics are not fully clean**, Chapter 7. The daily volume grows 10 times over the year. **Inference runs on one small GPU, and a 210m or a 610m encoder fits it.** The team spends the LLM budget on training data.

Table 1.1: Targets and constraints of the model. Quality is measured against the LLM topics, so the LLM topics set a ceiling for the model.

| Item | Scope | Target | Note |
|---|---|---|---|
| Recall, precision | Level 1 (17 topics) | $R \geq 0.90$, $P \geq 0.85$ | on the validation data |
| Recall, precision | Level 2 (106 topics) | $R \geq 0.85$, $P \geq 0.70$ | |
| Recall, precision | Level 3 (168 topics) | $R \geq 0.80$, $P \geq 0.60$ | looser as the topic count grows |
| Throughput | production | 20k articles/day by September | 200k/day by January |
| Hardware | production | one small GPU | a 210m or a 610m encoder fits |
| Languages | input | French first, multilingual later | |

## 1.4 Contributions of the internship

The work has 4 parts.

1. **Training data. The easy data set** (v2, Chapter 3): a random sample of the titles, the rare topics folded into their parent topic, and **1 article per title written and tagged by a small LLM**. It carried the model work for 2 months. **The gold data set and its real test set** (Chapter 3), built with the team in September: several LLM runs vote on each topic, and rules clean the data and drop the rarest topics.

2. **A model that knows the tree. HGCLR is a training method that teaches the encoder the tree**. For each article it keeps the words that its topics point to, and it trains the encoder to give the same output for the full article and for the kept words. **I rebuilt HGCLR with EuroBERT in place of BERT and with 4 changes**: attention pooling in place of a single token or a simple pooling, a deeper head, the ASL loss, and an attention block from the article to the topics before the pooling. **I compared 3 ways to make the output scores coherent**, which means a parent score at least as high as each child score. One of them is **a new averaging rule that moves a parent and its child toward each other**; it costs about as much as the maximum rule of C-HMCNN and scores higher (Chapter 4).

3. **Experiments on the easy data set. I built one platform to compare each choice of the project on the same footing**: encoders, model structures, losses, optimizer settings and coherence rules, each with several seeds, and **every run stored with its commit hash and its metrics** (Chapter 6).

4. **Analysis tools. A dashboard that shows the scores of a run per level and per topic, and lists the topics and the articles the model gets wrong**, so that an error can be traced to a topic, an article or a label (Chapter 7).

A short side study (Chapter 9) compares open rerankers for a nearby need of the product, thematic search.

## 1.5 Outline of the report

Chapter 2 gives the background: the topic tree, the task and the methods it borrows from, the encoders, the losses and the metrics. Chapter 3 describes the 3 data sets. Chapter 4 presents the model, from the baselines to HGCLR, my changes and the coherence rules. Chapter 5 states the environment and the protocol of the runs. Chapter 6 reports the results, first on the easy data set and then on the gold training set and the real test set. Chapter 7 analyses the errors. Chapter 8 lists the limits of the work. Chapter 9 reports the reranker study, and Chapter 10 ends with a personal review.

# Chapter 2

# Background

## 2.1 The IPTC Media Topics tree

**IPTC Media Topics** [4] **is the topic list of the news industry**, kept by the International Press Telecommunications Council. Each topic has a code, a name and a one-line definition in many languages. **The topics form a tree**: each topic has one parent, and the tree goes 6 levels deep. **The project uses the first 3 levels**: 17 topics on level 1, 120 on level 2 and 515 on level 3. The level 1 topics have no common root, so the tree is a forest of 17 trees. A press article gets a set of topics, and **a topic implies its parent topics up to level 1**: an article about a football match is about *football*, *competition* and *sport* at once.

## 2.2 Hierarchical multi-label classification

**The task is multi-label**: an article can have several topics, so the model gives one score per topic and a threshold turns each score into a yes or a no. **The task is hierarchical too**: the set of topics of an article contains the parents of each of its topics, so a child topic never comes without its parent. **A set of predictions is coherent when each parent scores at least as high as each of its children**; a threshold on coherent scores then keeps the tree intact.

**A flat model gives one score per topic and knows nothing of the tree**. It can say *football* and deny *sport*. **Hierarchy-aware models bring the tree in at 1 of 2 places**. Some put it into the model, so that the vectors of the topics or of the article carry the tree. Others put it at the output, with a rule that makes the scores coherent. **This report tries both places and compares them, alone and together**: HGCLR puts the tree into the encoder, and a coherence rule makes the scores coherent, at inference and, in one variant, in training as well.

**HiAGM [9] is an earlier model that puts the tree into the model**: a graph network turns the tree into one vector per topic, and each topic vector attends over the text to get its own view of it. The Graphormer label-attention baseline of Chapter 4 follows this design. **HGCLR [7] uses the topic vectors in training alone**, to build the positive view, and its paper reports higher scores than HiAGM on the 3 usual benchmarks.

HGCLR works as follows. **A graph encoder turns the topic tree into one vector per topic**. For each training article, **a generator uses these vectors and the article's topics to keep the tokens that speak for the topics and to mask the rest**; this masked copy is the positive view of the article. The encoder reads the article and its positive view, and **a contrastive loss pulls their 2 vectors together while pushing the other articles of the batch away**. The encoder thus learns which tokens carry the topics and keeps this knowledge at inference, where the model is a plain encoder with a head. Chapter 4 gives the blocks and the losses.

## 2.3 Encoders for French press

**An encoder is a transformer that reads a whole text and gives one vector per token**, where each vector carries the context of the token. Encoders of the BERT family are pre-trained on large text with masked tokens to fill in, then fine-tuned on the task at hand. Most of the articles here are French press articles of some length, and the product must handle other languages too, so **the encoder must read French well, handle several languages and accept long inputs**. Table 2.1 lists the 2 encoders I compared.

Table 2.1: The 2 encoders compared in this report. Sizes in parameters; input length in tokens.

| Encoder | Languages | Sizes | Trained for | Input length |
|---|---|---|---|---|
| EuroBERT | European languages | 210m, 610m, 2.1B | masked tokens | 8,192 |
| multilingual-e5-base | about 100 languages | 280m | retrieval | 512 |

**EuroBERT [1] is a recent encoder for European languages**, in 3 sizes, with the recipe of recent decoder models: rotary positions, no bias, and a long input window. Its tokenizer adds an end token and no start token, a detail that matters for the pooling in Chapter 4. **multilingual-e5 is an older multilingual encoder trained for retrieval**, so its vectors are made for comparing texts, and it reads short inputs.

## 2.4 Coherent predictions: C-HMCNN and the max rule

**C-HMCNN [3] makes the output of any multi-label model coherent with one rule, the max rule**. Let $p_j$ be the score of topic $j$ and $D_j$ the set made of $j$ and all its descendants. The output score of $j$ is

$$\mathrm{MCM}_j = \max_{k \in D_j} p_k. \tag{2.1}$$

A parent then scores at least as high as each child, and a threshold on these scores respects the tree.

**The rule alone has a flaw in training**: a child with a wrong high score lifts its parent, and the loss then pushes the parent down instead of the child. C-HMCNN therefore trains with a loss where, **for a topic the article has, the maximum runs over the descendants it has too**:

$$L_j = -y_j \ln\Big(\max_{k \in D_j} y_k\, p_k\Big) - (1 - y_j) \ln\big(1 - \mathrm{MCM}_j\big). \tag{2.2}$$

The gradient then reaches the topic that caused the error. Chapter 4 uses this rule and derives from it the avg-max rule of the final model.

## 2.5 Losses for imbalanced multi-label data

**The usual loss for multi-label classification is binary cross-entropy**, one yes-or-no loss per topic. With hundreds of topics and a few topics per article, almost all the terms are negatives, and **the model learns to say no**. **The first fix weights the positive term of each topic by how rare the topic is**, with a cap on the weight. **The second is the asymmetric loss, ASL [5]**, and Figure 2.1 shows its idea. The top plot draws the loss on one negative against the score the model gives it. Cross-entropy keeps a loss on each negative down to a score of 0, so the many easy negatives add up and drive the training. Focal loss scales this loss down for a negative with a low score. **ASL goes one step further and shifts the curve to the right by a margin** $m$, so a negative with a score below $m$ costs nothing. The positives keep the full loss. The bottom plots show the result on MS-COCO, where $p_t$ is the model's confidence

in the right answer. With cross-entropy the negatives reach a high confidence within the first steps while the positives stay below. **With ASL the 2 curves meet**. ASL has 3 settings, one exponent per side and the margin, and with all 3 at 0 it is binary cross-entropy again. Chapter 4 states the values used.

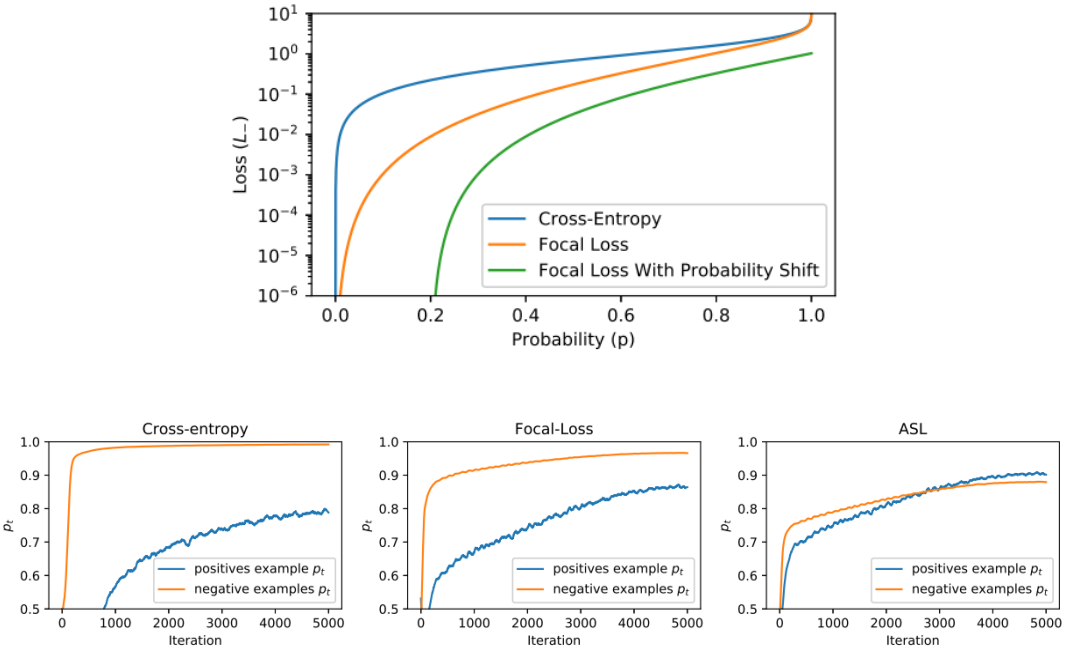



Figure 2.1: The idea of ASL, 2 figures from Ridnik et al. [5]. Top: the loss on a negative against its score, for cross-entropy, focal loss, and focal loss with the margin $m$. Bottom: the mean confidence in the right answer, $p_t$, of the positives and of the negatives during training on MS-COCO, with cross-entropy, focal loss and ASL.

## 2.6 Evaluation

The model gives one score per topic, and I read the scores in 2 ways. With a threshold, each score becomes a yes or a no, and the usual counts follow: **recall, the share of the article's topics the model found, and precision, the share of the model's topics that are right**. F1 is their harmonic mean. With $TP$ the topics found, $FP$ the topics predicted but wrong and $FN$ the topics missed,

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}. \tag{2.3}$$

Without a threshold, **average precision (AP) ranks the scores and rewards a model that puts the right topics first**, whatever the threshold. It walks down the ranked list and sums the precision at each point where a right topic appears, weighted by the recall gained there:

$$AP = \sum_k \left(R_k - R_{k-1}\right) P_k, \tag{2.4}$$

where $P_k$ and $R_k$ are the precision and the recall of the first $k$ scores.

**Each of these comes in 2 flavours**. **Micro pools all decisions of all articles into one count**, so the frequent topics dominate. **Macro computes the score per topic and takes the mean**, so a rare topic counts as much as a frequent one. For precision, with $L$ topics

and $TP_j$, $FP_j$ the counts of topic $j$,

$$P_{\text{micro}} = \frac{\sum_{j=1}^{L} TP_j}{\sum_{j=1}^{L} (TP_j + FP_j)}, \qquad P_{\text{macro}} = \frac{1}{L} \sum_{j=1}^{L} P_j, \tag{2.5}$$

and the same for recall, F1 and AP.

**The targets of Table 1.1 are recall and precision per level**, so the tables report them per level. For a target recall $R^*$, **the threshold $\tau$ goes to the point where the recall reaches the target, and the precision there says whether the target holds**:

$$P@R^* = P(\tau^*), \qquad \tau^* = \max\{\tau : R(\tau) \geq R^*\}. \tag{2.6}$$

2 more numbers serve the tree. **Recall at $k$ is the share of an article's topics $y_i$ among its $k$ highest scores**, averaged over the $N$ articles:

$$R@k = \frac{1}{N} \sum_{i=1}^{N} \frac{|\, y_i \cap \text{top}_k(i) \,|}{|\, y_i \,|}. \tag{2.7}$$

**The break value $B$ measures the breaks of the tree on the raw scores.** For each parent $p$, take each child $c$ that scores above it and square the gap $p_c - p_p$. Sort the gaps under one parent from large to small, and give the $r$-th the weight $1/r$. $B$ is the sum over the parents, averaged over the articles:

$$B = \frac{1}{N} \sum_{i=1}^{N} \sum_{p} \sum_{r} \frac{1}{r}\, g_r^2, \qquad g_r \text{ the } r\text{-th largest gap under } p \text{ in article } i. \tag{2.8}$$

**The square makes one real break count more than many small ones, and the weights $1/r$ stop one low parent from counting once per child. $B = 0$ means coherent scores.** The run index of the old platform records it as `broken`.

# Chapter 3

# Data: 3 data sets

## 3.1 Starting point

**At the start of the internship no training data existed for the task**: no article with its 3 levels of topics. The team had 2 things. **The first was a set of titles** from its thematic search work, each title with a level 1 and a level 2 topic and no article text. **The second was a scrape of real French press articles**, with no topics yet. **Copyright keeps real articles out of training**, so this scrape could serve as test data alone. **3 data sets followed**, Table 3.1: an easy data set to try the methods, a gold training set, and a real test set.

Table 3.1: The 3 data sets. The easy data set is mine; the gold training set and the real test set are the team's.

| | Easy data set | Gold training set | Real test set |
|---|---|---|---|
| Built by | me, August | the team, September; pruned 21 September | the team, September; trimmed 5 October |
| Text | LLM-written from a title | LLM-written from an LLM-written seed title | real French press, scraped in 2026 |
| Writer LLM | gpt-5.4-nano | **gpt-5-mini** | none |
| Topics given by | the same LLM, 1 vote | 3 LLMs, all votes kept | the same 3 LLMs |
| Languages | French | 2/3 French, 1/3 English | French |
| Articles | 10,798 | 58,527 (59,495 before pruning) | 9,701 (10,000 before trimming) |
| Topic list | 294 distinct; 186 kept by the model | 17 level 1, 120 level 2, 192 kept level 3 | the same list |
| Use | compare methods, Sections 6.1 to 6.4 | train the model, Section 6.5 | measure the model, never train |

## 3.2 The easy data set

**My first design**, in the first week of August, had an LLM read real articles of one topic, write a profile of them, and generate articles from that profile. **A profile is an average, and the model needs many kinds of writing per topic, so I dropped it** within the week. **The second design**, which produced the data set within the same week, **starts from the titles instead**. The team had generated these titles earlier as **niche titles: unusual kinds of news within each topic**, not the average news of the topic, so that a model learns the edges of a topic and not its centre alone. **The design runs in 4 steps. First I picked a sample of the titles**, spread over the level 1 and level 2 topics, with a cap per pair of topics so that no topic floods the sample. **Second, a small LLM wrote one press article from each title.**

**Third, the same LLM read the article and tagged it with a tree of topics down to level 3**; a tag outside the tree, or under the wrong parent, went back to the LLM, and after a few tries the invalid part was cut. I dropped the articles whose level 1 topic did not match the title's. **Fourth, I folded the long tail**: a topic with too few articles went into its parent topic, so that the data set holds topics the model can learn. One seed fixes the split into training and validation data, and **the model's topic list drops the topics that appear in less than 1 % of the training articles**.

## 3.3 The gold training set

**The gold training set is the team's work**, built in September; I took part in the design discussions. **It keeps the idea of the easy data set, LLM-written articles, and fixes its 2 weaknesses**: 1 LLM as both writer and judge, and topics with no articles. **The team cut the tree to the topics the product needs. For each kept topic 1 LLM wrote seed titles and a stronger LLM wrote 1 article per title**, in French and in English, in several press formats and lengths. **The topics of an article come from an LLM in 2 stages**: it first picks the level 1 topics, and then receives the subtrees of those topics alone to pick the deeper ones, which keeps the prompt short. Several LLMs tag each article this way, and where they disagree on a topic, a stronger LLM judges it.

A later pass pruned the set, since the 3 voters tag too many topics. A judge LLM of another kind answered one yes-or-no question per level 1 topic; a rejected topic left the article with its children, and an article the judge rejected as a whole left the set. The set before pruning stays on the Hub.

## 3.4 The real test set

The real test set measures the model and never trains it. The team scraped French press in 2026 and drew a sample balanced over the level 1 topics, so that each level 1 topic has about the same number of articles; religion, the rarest, has fewer. The same 3 LLMs tagged the articles from their body, with the same votes. A later pass trimmed the set: pages with no article body behind a paywall or a cookie wall, which the voters had tagged from the wall text, and duplicate copies of one article. A lede cut by a paywall stays, since this is how scraped news arrives. The topics of this set are votes, not ground truth: no human-tagged set exists yet, which Chapter 7 returns to.

## 3.5 Purpose and limits

**Each data set has one job.**

- **The easy data set explores the effect of each method and each change**, Sections 6.1 to 6.4. It is easy in 2 ways: **the LLM that wrote an article tagged it as well**, so the topics fit the text, and **the rare topics are gone**.
- **The gold training set trains the model for the product**, with the full topic list.
- **The real test set measures the model on real articles.**

**2 limits hold for the 3 data sets. The topics come from LLMs, not from people**, so a score measures how far the model agrees with the LLMs. And **the generated articles differ from real press** in which topics appear and in their shares, a gap that Chapter 7 measures.

# Chapter 4

# Method

## 4.1 Model overview

**The model takes 2 inputs**, shown on the top row of Figure 4.1: **the IPTC topic tree**, and **an article with its title and text**. In training the article also brings its topics $y$. The topic side of the figure works on the tree. Each topic gets a vector: the EuroBERT embedding of its name plus a learned vector of its own. The IPTC topics form a forest: one tree per level 1 topic. A Graphormer then mixes the vectors within each tree, so that each topic vector carries the whole tree it belongs to. The result is $G$, one vector per topic. $G$ **is learned during training and does not change at inference**.

**The article side is the solid chain in the middle**. **The input is one string**, the title after the word `title:` and the text after the word `article:`, so the encoder knows which part is which. The tokenizer cuts this string into tokens, and the embedding table turns them into vectors $e$. The encoder $g_\theta$, a pre-trained EuroBERT, reads $e$ and outputs **one vector per token**. The article-to-topic attention scores each token vector against all the topics in $G$ and **masks the unimportant tokens**. A small feed-forward layer then transforms each token vector on its own. Attention pooling merges the token vectors into **one vector $z$ for the article**. The head maps $z$ to **one score per topic**. The avg-max rule makes the scores coherent: **a parent topic scores at least as high as each of its children**. **Each topic whose score passes the threshold becomes a tag of the article**. **This chain is the whole model at inference**.

**Training adds the positive view**. The positive-sample generator compares each token embedding with $G$ and masks the tokens that are unimportant for the article's topics $y$. This masked copy $\hat{e}$ goes through the same chain as the article and gives **a second vector $\hat{z}$ and a second set of scores**. The training loss then asks 2 things: **the scores of the article and of the positive view must match $y$, and $z$ and $\hat{z}$ must be close**. Section 4.3 gives the losses.

Table 4.1 sums up the blocks. **HGCLR [7] gives the frame**: the topic side, the generator and the contrastive loss, on top of a BERT encoder, here EuroBERT. Inside this frame **I replaced 4 blocks with my own**: the article-to-topic attention with its feed-forward layer, the pooling, the head and the classification loss. On the output side **I added avg-max, a coherence rule derived from C-HMCNN [3]**. The rest of this chapter presents these parts one by one, starting with the baselines I compared the model against.

## 4.2 Baselines

The work started on the easy data set with the simplest model: a frozen encoder and a head on top. This model is cheap enough to run many times, so I used it first to compare encoders, different BERT models and different sizes. The models that followed add one thing each: training the encoder, a cheaper way to train it, and a head that knows the tree. All of them share the

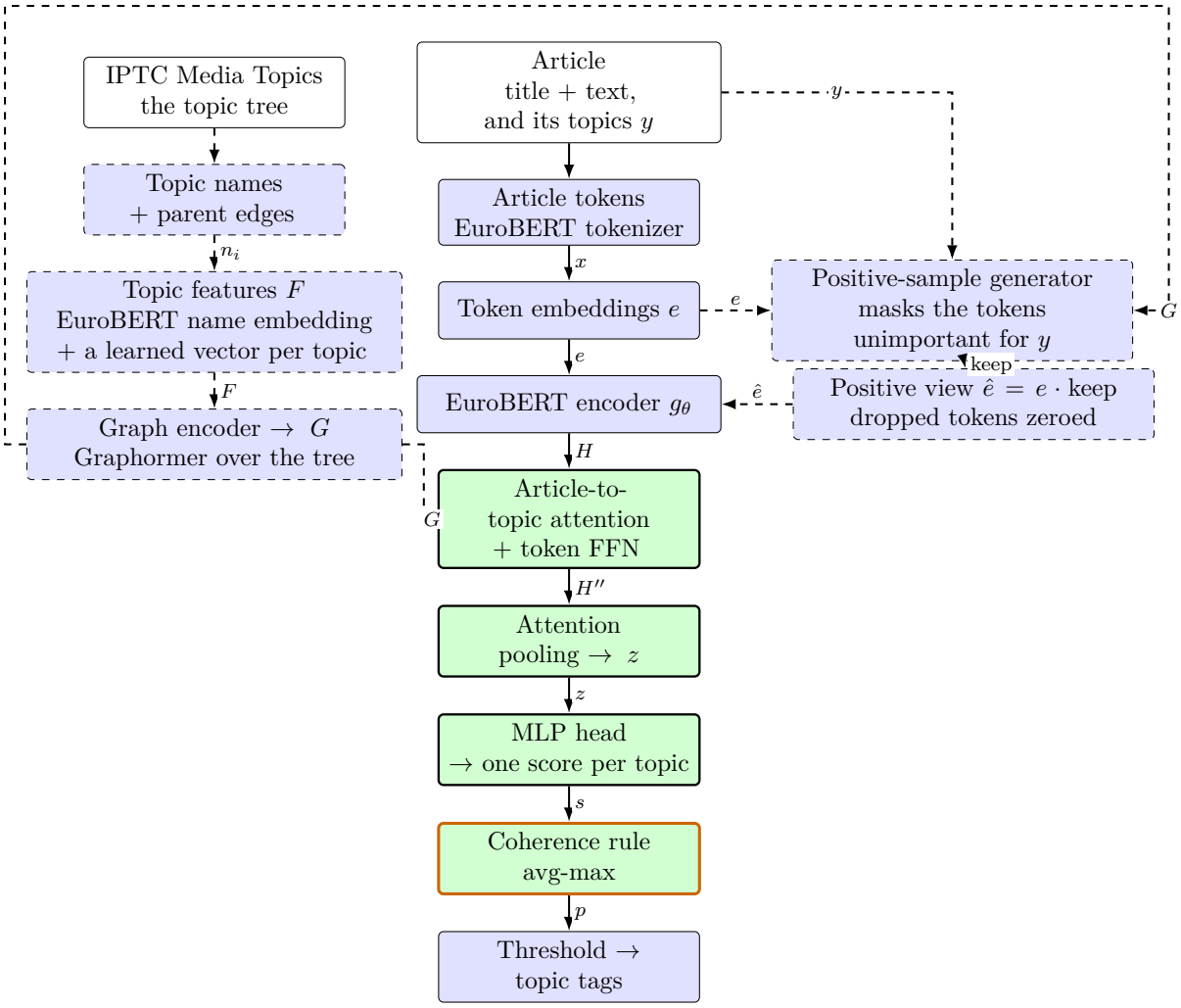



Figure 4.1: The model: the data flow from the inputs on the top row to the topic tags. White: inputs. Blue: HGCLR blocks. Green: our changes. Green with an orange border: avg-max, our variant of the C-HMCNN rule. Solid: the inference path, one pass per article. Dashed: training only. The letters on the arrows are the symbols of the equations in Section 4.3; the positive view $\hat{e}$ follows the same chain as the article and gives $\hat{z}$ and $\hat{s}$. The losses are in Section 4.3; the full graph with tensor shapes is in Appendix A.

Table 4.1: The blocks of the model in the order of Figure 4.1, where each one comes from, and what it does.

| Block | Origin | What it does |
|---|---|---|
| Topic features $F$ | HGCLR | one vector per topic: the EuroBERT embedding of the topic name plus a learned vector |
| Graph encoder $G$ | HGCLR | a Graphormer mixes the topic vectors within each tree of the forest |
| Encoder $g_\theta$ | EuroBERT | a pretrained BERT; reads the article tokens and, in training, the positive view |
| Positive-sample generator | HGCLR | in training, keeps the tokens that match the article's topics $y$ and masks the unimportant tokens |
| Article-to-topic attention + token FFN | ours | scores each token against all the topics in $G$ and masks the unimportant tokens, then a small feed-forward layer on each token |
| Attention pooling | ours | merges the token vectors into one vector $z$ per article |
| Head | ours | 3 linear layers from $z$ to one score per topic |
| Classification loss | ours | asymmetric loss (ASL) on the scores |
| Contrastive loss | HGCLR | pulls $z$ and $\hat{z}$ together |
| Coherence rule | C-HMCNN, ours | avg-max: a parent topic scores at least as high as each child; our variant of the C-HMCNN rule |

split, the topic set, the loss family and the metrics. Table 4.2 lists them; the results are in Chapter 6.

Table 4.2: The baselines, in the order they were tried on the easy data set. All share the split, the topic set, the loss family and the metrics of the HGCLR runs.

| Step | Model | Encoder | What trains | Question it answers |
|---|---|---|---|---|
| 1 | Frozen + MLP head | EuroBERT 210m, 2.1B; multilingual-e5. Frozen, one pass, vectors stored | the head | which encoder, and how far a frozen encoder goes |
| 2 | Fine-tune | EuroBERT, all layers or the top layers trained | encoder and head | what training the encoder adds, and how much of it |
| 3 | LoRA | EuroBERT, weights fixed, low-rank adapters | adapters and head | fine-tune quality at a fraction of the memory |
| 4 | Graphormer label attention | EuroBERT, frozen or trained | Graphormer, topic attention, (encoder) | whether a head that knows the tree helps |

### 4.2.1 Frozen encoder and an MLP head: choosing the encoder

**The first model keeps the encoder as it is**. It runs each article through the encoder once and pools the token vectors into one vector per article. **The pooling is a fixed rule: the mean and the maximum over the tokens**, side by side. Figure 4.2 shows the chain. **I store these vectors on disk**, so the encoder never runs again. **The head is a standard classifier**, a small MLP trained on the stored vectors with binary cross-entropy. **It is the baseline for everything that follows**: the choice of encoder, the architectures, the learning rates and the schedules. Training the head takes minutes, so I can run many settings and repeat each one with several seeds.

Because the encoder runs once, **this model is the right place to compare encoders**. I

compared **2 models, EuroBERT and multilingual-e5, and 2 sizes of EuroBERT, 210m and 2.1B**. The encoders differ in the length they accept, the prefix their input needs and the pooling their weights were trained for, so I wrote these facts into one table in the code, and one training loop reads it. EuroBERT-2.1B scored higher, but a full fine-tune of it costs far more time and memory, so **EuroBERT-210m became the encoder for the main runs**. If a change made no clear difference on 210m, I tried it again on the bigger model. Chapter 6 gives the numbers.

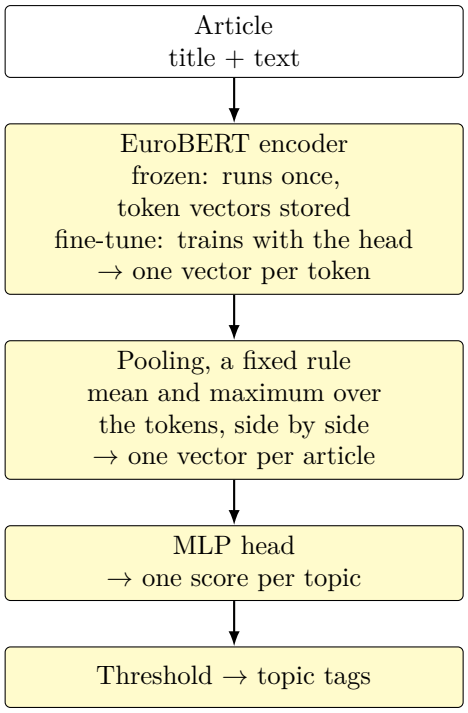



Figure 4.2: The frozen and the fine-tuned baselines share one chain. Frozen: the encoder runs once and its token vectors are stored. Fine-tune: the encoder trains with the head. White: input. Yellow: the blocks of the baselines.

### 4.2.2 Training the encoder: warm-up, learning rates, top layers, LoRA

**The second model trains the encoder as well as the head**, and it beats the frozen model. The head starts random, so its first gradients are noise, and noise pushed into a pretrained encoder erases what it learned, a failure known as **catastrophic forgetting**. 2 standard settings prevent it: **the encoder stays frozen for the first epochs while the head trains**, which I call the warm-up, and then **trains with a lower learning rate than the head**. **This full fine-tune model costs more than the frozen one in 2 ways**. The optimizer, AdamW, keeps 2 running averages for each weight of the encoder, so **the training state takes 3 times the memory of the encoder itself**. And **each run saves the whole encoder to disk**, where the frozen model saves a small head.

**2 cheaper options sit between the frozen encoder and the full fine-tune**. The first **trains the top layers of the encoder and freezes the lower ones**. The frozen layers keep their pretrained features, and AdamW holds no state for them, so the run is lighter. Early in the exploration I trained 2 or 3 top layers, without a record, and saw little change from the frozen model. Later, with HGCLR, **training the top 6 layers cut the training time and scored above the frozen model**. I have no clean comparison with the full fine-tune: the 2 runs used different settings for the positive samples, so the gap between them has 2 causes. I kept the option for cheap test runs; **the final model trains the whole encoder**.

**The second option is LoRA**. The pretrained weights stay fixed. Each attention projection gets a pair of small low-rank matrices, and **the gradient reaches these matrices and the head**, under 1 % of the parameters. I tried it in the early exploration, without a record: **LoRA brought no clear gain over the frozen model**, and I dropped it.

Chapter 6 compares the options.

### 4.2.3 Graphormer label attention

**The third model is the first one that sees the tree**. Figure 4.3 shows its 2 sides. The topic side starts from the tree. **Each topic gets a vector from the frozen EuroBERT embedding of its name and definition**, plus a learned vector for its number of neighbours. **A Graphormer then runs attention between the topics**, and the attention score of 2 topics carries a learned bias set by their distance in the tree. The article side runs the article through EuroBERT. **The 2 sides meet in the label attention of HiAGM-LA [9]**: each topic attends over the tokens of the article and gathers its own view of it, and **a scorer per topic turns that view into the topic's score**. The encoder can stay frozen or train with the rest.

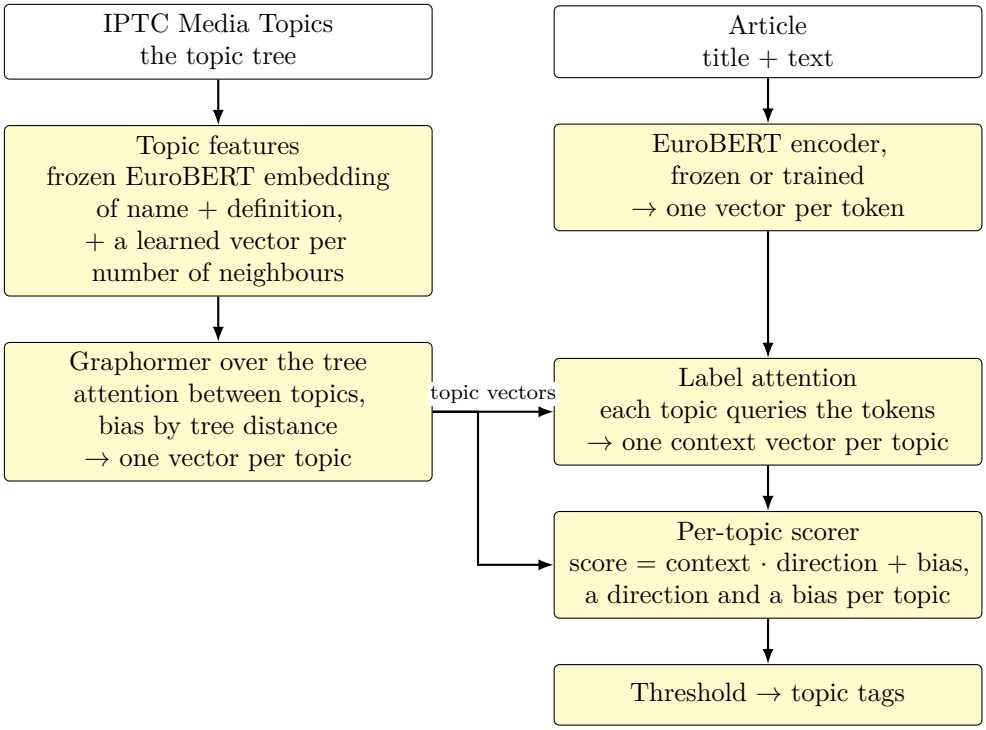



Figure 4.3: The Graphormer label-attention baseline. The topic side on the left and the article on the right meet in the label attention, where each topic reads the article for itself. White: inputs. Yellow: the blocks of this baseline.

A first version scored all topics with one shared scorer and gave each topic the same score; **the scorer per topic fixes this**.

### 4.2.4 Several seeds: the spread and the ensemble

**A small gain may be noise**. The seed sets the random start of the head, the dropout and the batch order, and 2 seeds of the same setting can differ by more than the gain. So **I ran each setting with several seeds**: more for the cheap models, fewer for the models where each seed is a full fine-tune. **A change counts as real when it beats the spread of the seeds**. The seeds give an ensemble too: **the mean of their probabilities, which scores much higher**

**than any single seed**. For the frozen model this is free, since each seed is one more head on the same stored vectors. Chapter 6 reports both.

## 4.3 The model block by block

At inference the model sees the article alone. The topic tree can help it then in one way: **the encoder must have learned the tree during training**. Hierarchy-guided contrastive learning (HGCLR) [7] does this by **making a positive view of each training article: a copy that keeps the tokens about the article's topics and masks the rest**. A generator reads the article's topics $y$ and chooses these tokens. **The encoder reads the article and this positive view with the same weights**, and **a contrastive loss pulls together the 2 output vectors, one for the article and one for its positive view**. **The encoder learns to find the topic-bearing tokens on its own**, and at inference it does so without $y$. This section walks through the blocks of Figure 4.1 in the order of the data flow. **The topic side, the generator and the contrastive loss come from HGCLR; the article-to-topic attention, the pooling, the head and the classification loss are mine**. Table 4.3 lists the settings of the blocks.

Table 4.3: Settings of the blocks in the run configuration. The HGCLR rows follow the paper and its reference code; the other rows are our blocks.

| Block | Origin | Setting | Value |
|---|---|---|---|
| Graph encoder | HGCLR | Graphormer layers | 1 |
| | | attention heads | 12 |
| Generator | HGCLR | attention heads | 8 |
| | | keep threshold $\gamma$ | 0.02 |
| | | Gumbel temperature $\tau$ | 1 |
| Article-to-topic attention | ours | attention heads | 8 |
| | | token FFN hidden width | 768 |
| Attention pooling | ours | heads | 12 |
| | | MLP hidden width | 128 |
| Head | ours | hidden width | 1024 |
| | | layers | 3 |
| | | dropout | 0.2 |
| Classification loss (ASL) | ours | $\gamma_+$ | 0 |
| | | $\gamma_-$ | 1 |
| | | margin $m$ | 0.05 |
| Contrastive loss | HGCLR | temperature $\tau_{con}$ | 1 |
| | | weight $\lambda$ | 0.1 |

### 4.3.1 Topic features and the graph encoder

**Each topic $i$ starts as a vector $f_i$ with 2 parts**. **The first part is the mean of the EuroBERT token embeddings of the topic name**, so the graph encoder starts with basic information on each topic. **The second part is a learned vector $E_i$ of the topic's own**, which training moves as it needs to tell topics apart:

$$f_i = E_i + \text{name}_i, \qquad \text{name}_i = \frac{1}{|n_i|} \sum_{w \in n_i} \text{embed}(w), \tag{4.1}$$

where $n_i$ is the set of tokens in the name of topic $i$, $w$ runs over these tokens, embed($w$) is the EuroBERT embedding of token $w$, and $f_i$ is row $i$ of $F$. I set the start scale of $E$ to the scale of the token embeddings.

**The graph encoder is one Graphormer layer** [8], as in HGCLR: attention among the topics, where **the attention score of a pair of topics gets a learned bias from their distance in the tree** and from the edges on the path between them:

$$a_{ij} = \frac{(W_Q f_i) \cdot (W_K f_j)}{\sqrt{d}} + b_{\mathrm{dist}(i,j)} + c_{ij}, \tag{4.2}$$

with $b$ one learned value per distance and $c_{ij}$ the mean of a learned value per edge on the path from $i$ to $j$. A softmax over $j$ and a weighted sum of the $f_j$ follow, as in any attention layer. **The output is $G$, one vector per topic that mixes in all the other topics of its tree**, weighted by distance; the tree knowledge itself sits in the learned biases and topic vectors, and $G$ is recomputed from them at each step. In HGCLR, $G$ serves the generator alone. **In our model the article-to-topic attention of Section 4.3.3 reads $G$ too, at inference as well**.

### 4.3.2 Positive-sample generator and the 2 views

**The generator decides which tokens of a training article survive into the positive view**, as in HGCLR. **It works on the token embeddings $e_i$, before the encoder**. The equations follow the paper. Each token $x_i$ is a query and each topic $j$, with its row $G_j$, is a key:

$$q_i = e_i W_Q, \qquad k_j = G_j W_K, \qquad A_{ij} = \frac{q_i k_j^\top}{\sqrt{d_h}}. \tag{4.3}$$

A softmax over the topics would give the probability that token $x_i$ belongs to topic $j$. **HGCLR replaces it with a Gumbel-softmax, which adds noise and keeps the sampling differentiable**:

$$P_{ij} = \mathrm{gumbel_softmax}(A_{i1}, A_{i2}, \ldots, A_{ik})_j. \tag{4.4}$$

**The sum over the article's topics $y$ gives the probability of the token under its topic set, and a threshold $\gamma$ builds the positive view**:

$$P_i = \sum_{j \in y} P_{ij}, \qquad \hat{e}_i = \begin{cases} e_i & \text{if } P_i > \gamma \\ 0 & \text{else} \end{cases}. \tag{4.5}$$

The threshold has no gradient, so **the generator passes the hard choice forward and the gradient of $P_i$ backward**. **In HGCLR this is the one path that trains the topic side**. The noise makes the choice random, so **the same article gets a different positive view at each epoch**.

In the positive view $\hat{e}$, **a kept token keeps its embedding and a dropped token becomes a zero vector at its position**. **I run the encoder once: the article and its positive view go into one batch of $2B$ rows**, which works because EuroBERT accepts embeddings as its input. From the encoder on, **the 2 views go through the same blocks with the same weights**, and come out as the vectors $z$ and $\hat{z}$ and the scores $s$ and $\hat{s}$.

### 4.3.3 Article-to-topic attention and token FFN

The generator highlights the tokens that carry the article's topics, but it reads $y$ to do so, and $y$ exists in training alone. **I added a block that does the same at inference, where there is no $y$: the article-to-topic attention**. It sits after the encoder and **scores each token state $H_t$**, the encoder's output for token $t$, **against the graph-aware topic features $G$** of the graph encoder, with the same attention as the generator:

$$q_t = H_t W_Q, \qquad k_j = G_j W_K, \qquad S_{tj} = \sigma\Big(\frac{q_t k_j^\top}{\sqrt{d_h}}\Big), \tag{4.6}$$

averaged over the heads. **A sigmoid replaces the softmax, and the maximum over all topics replaces the sum over** $y$, so the block runs without $y$:

$$r_t = \max_j S_{tj}, \qquad H'_t = H_t + \alpha\, r_t\, H_t. \tag{4.7}$$

The block drops no token; **it scales up a token that matches a topic**, with one learned scalar $\alpha$ that starts at 0. **A feed-forward layer on each token follows**, with the same zero start:

$$H''_t = H'_t + W_2\,\mathrm{GELU}\big(W_1\,\mathrm{RMSNorm}(H'_t)\big), \qquad W_2 = 0 \text{ at the start.} \tag{4.8}$$

**Both blocks begin as the identity**, so the model starts as the one without them.

### 4.3.4 Attention pooling

HGCLR takes the vector of the first token as the article vector, the usual BERT choice. **EuroBERT has no such token**: its tokenizer adds an end token and nothing in front. The frozen baseline of Section 4.2.1 pooled with a fixed rule, the mean and the maximum over the tokens. **The final model pools with attention**, the gated attention pooling of the torchmil library [6]: **a small MLP gives each token an attention value, a softmax over the tokens normalises the values, and the article vector is the weighted sum of the token states**:

$$\mathbf{f} = \big(\tanh(H''\mathbf{W}_1) \odot \mathrm{sigm}(H''\mathbf{W}_2)\big)\,\mathbf{w}, \qquad z = H''^{\top}\,\mathrm{Softmax}(\mathbf{f}) = \sum_{t=1}^{T} s_t\, H''_t, \tag{4.9}$$

where $H''$ holds one token state $H''_t$ per row, $\mathbf{f} \in \mathbb{R}^T$ are the attention values, $s_t$ is the normalised score of token $t$, and $\mathbf{W}_1$, $\mathbf{W}_2$ and $\mathbf{w}$ are the weights of the MLP. With several heads, $\mathbf{w}$ has one column per head, and **each head pools its own slice of** $H''$ **with its own scores**. The softmax skips the padding tokens. **The article vector is a weighted mean of the token states, where the model chooses the weights** and each head can look at different tokens. This block gave the largest single gain of my 4 changes on the easy data set, on the frozen baseline and inside HGCLR alike. A dropout on $z$ follows, and $z$ feeds both the head and the contrastive loss.

### 4.3.5 Head

HGCLR maps $z$ to the scores with one linear layer. **Our head is a small MLP of 3 linear layers**, with a ReLU and a dropout after the first 2:

$$s = W_3\,\mathrm{ReLU}\big(W_2\,\mathrm{ReLU}(W_1 z)\big), \tag{4.10}$$

biases and dropouts left out. The last layer has no activation: $s$ **is one raw score per topic**, and the sigmoid and the coherence rule come after. **One head scores the article and the positive view**. The frozen baseline used a head of the same shape, and I kept it in the final model.

### 4.3.6 Classification loss

**The classification loss compares** $p = \sigma(s)$ **with the article's topics** $y$, one term per topic. The baselines used binary cross-entropy with a capped positive weight per topic, against the imbalance of Section 2.5. **The final model uses the asymmetric loss (ASL) [5]**:

$$p^- = \max(p - m, 0), \qquad L^+ = (1-p)^{\gamma_+}\log p, \qquad L^- = (p^-)^{\gamma_-}\log(1-p^-), \tag{4.11}$$

$$L^C = -\frac{1}{B} \sum_{\text{articles}} \sum_{j=1}^{L} \left[ y_j L_j^+ + (1 - y_j) L_j^- \right]. \tag{4.12}$$

**The margin $m$ shifts the probability of a negative down and drops the easy negatives from the loss**; the exponent $\gamma_-$ dampens the ones that remain; $\gamma_+$ does the same on the positives. Figure 2.1 in Section 2.5 shows the effect. With all 3 settings at 0 the loss is binary cross-entropy. I swept them on the frozen baseline, without a detailed study. **ASL scored higher than binary cross-entropy, and the setting with a light focusing on the negatives, a small margin and no focusing on the positives scored best**; Table 4.3 gives the values, and Table 6.3 in Section 6.2 lists the sweep. The same loss runs on $s$ and on $\hat{s}$.

### 4.3.7 Contrastive loss and the total loss

**The contrastive loss pulls $z$ and $\hat{z}$ together and pushes the other articles of the batch away**. It is the NT-Xent loss of SimCLR [2], as in HGCLR. A small projection, 2 linear layers with a ReLU between them, maps each of the $2B$ vectors to a vector $c$ and scales it to unit length, so that a dot product is a cosine. **For each vector $m$ the loss is a softmax over the other $2B-1$ vectors, where the partner $\mu(m)$, the other view of the same article, must win**:

$$L^{con} = -\frac{1}{2B} \sum_{m=1}^{2B} \log \frac{\exp\left(\text{sim}(c_m, c_{\mu(m)})/\tau_{con}\right)}{\sum_{i \neq m} \exp\left(\text{sim}(c_m, c_i)/\tau_{con}\right)}. \tag{4.13}$$

Each article has one positive pair and $2B-2$ negatives in the batch. **The gradient of this loss flows back through both views into the encoder**. The positive view holds the tokens that speak for the topics and masks the rest, so to bring $z$ close to $\hat{z}$ **the encoder learns to build $z$ from these tokens**. The projection exists for this loss alone, and inference drops it.

**The total training loss adds the classification loss on both views, with the same $y$, and the contrastive loss weighted by $\lambda$**:

$$L = L^C(s, y) + L^C(\hat{s}, y) + \lambda L^{con}. \tag{4.14}$$

HGCLR uses binary cross-entropy for $L^C$; our model uses the asymmetric loss of Section 4.3.6. I kept $\lambda$ at the value the paper reports for its 2 data sets, 0.1, and did not re-tune it after the change of loss. **At inference the generator, the positive view and the contrastive loss disappear**; the model is the solid chain of Figure 4.1.

## 4.4 Coherence rules

Section 2.2 defined a coherent prediction: each parent scores at least as high as each of its children. The head of Section 4.3.5 has no sense of the tree, so **a rule has to make the scores coherent**. **One setting picks the rule, and with it where the rule runs**: at inference alone, as a step after the sigmoid, or in training as well, inside the loss. Table 4.4 lists the 5 settings I compared; Chapter 6 gives their scores.

### 4.4.1 Rules at inference

**A rule at inference takes the scores $p$ of one article and returns coherent scores**, and the threshold then runs on these. **The max rule is the one of C-HMCNN**, Equation 2.1: the score of a topic becomes the maximum over its subtree. It raises scores and lowers none, so it adds predictions and removes none; **recall goes up and precision goes down, by construction**.

**The avg-max rule is my variant**. Instead of lifting the parent to the child, **it moves both to their mean**: the parent goes up as much as the child goes down. It walks the tree

Table 4.4: The 5 coherence settings. The rule in training sits inside the classification loss; the rule at inference runs on $p = \sigma(s)$ before the threshold.

| Setting | Rule in training | Rule at inference | Origin |
|---|---|---|---|
| none | none | none | flat baseline |
| hard_max | none | max | C-HMCNN, as a step after the sigmoid |
| hard_avg_max | none | avg-max | ours, as a step after the sigmoid |
| chmcnn | max | max | C-HMCNN |
| chmcnn_avg_max | avg-max | avg-max | ours; the final model |

from the deepest level up; inside a level it visits the topics in rising order of score; for each topic it climbs its ancestors, nearest first, and **when an ancestor scores below the topic, both take their mean**:

$$p_k \leftarrow \frac{p_k + p_j}{2}, \qquad p_j \leftarrow \frac{p_k + p_j}{2} \qquad \text{when } k \text{ is an ancestor of } j \text{ and } p_k < p_j. \tag{4.15}$$

In code, for one article:

```
for each level, from the deepest to the top:
    for each topic j of the level, in rising order of p[j]:
        for each ancestor k of j, nearest first:
            if p[k] < p[j]:
                p[k] = p[j] = (p[k] + p[j]) / 2
```

An ancestor rises and a topic falls, and no step does the reverse, so **after the pass each parent scores at least as high as each child, and the violation rate is 0 at any threshold**. Table 4.5 runs the rule on one chain. **The max rule lifts the parents up to a wrong high child; the avg-max rule brings that child down as much as it lifts the parents**, so the sum of the scores stays the same. The reason to trust the parent is the data: **a parent has the articles of all its children, so it has more training data than any one child and scores with more reliability**. The max rule hands the parent the child's score; the avg-max rule weighs the 2. **The rule sorts the topics of a level by score so that the highest child meets its parent last**; the parent arrives at that meeting lifted by the lower children, and so within one subtree the order of the topics holds.

Table 4.5: The 2 rules on one chain root → A → a. The avg-max rule averages (A, a), then (root, a), then (root, A).

| | root | A | a |
|---|---|---|---|
| scores $p$ | 0.30 | 0.20 | 0.90 |
| max rule | 0.90 | 0.90 | 0.90 |
| avg-max rule | 0.4875 | 0.4875 | 0.425 |

### 4.4.2 Rules in training

A rule at inference repairs the scores and teaches the model no part of the tree. **C-HMCNN puts the rule inside the loss**. The loss of Section 4.3.6 then reads a corrected score $p_j$ in place of the raw score $h_j = \sigma(s_j)$:

$$p_j = y_j \max_{k \in D_j} y_k \, h_k \; + \; (1 - y_j) \max_{k \in D_j} h_k, \tag{4.16}$$

with $D_j$ the subtree of topic $j$, as in Equation 2.1. **For a topic the article has, the maximum runs over the article's own topics. For a topic it has not, the maximum runs over the**

**whole subtree.** A wrong high child thus raises the loss of its parent, and the gradient reaches the child. This is the setting chmcnn, Equation 2.2 with ASL in place of the cross-entropy. **The setting chmcnn_avg_max puts the avg-max rule of Section 4.4.1 in place of each maximum.** A soft value between 0 and 1 counts as a negative here.

**A model trained with a rule in the loss predicts with the same rule at inference**; its raw scores are not coherent on their own. Such a run converges slower than a flat run and needs more epochs. On the first version of the gold training set, Table 6.7, **the max rule in training lowered the scores**. At inference the max rule lowered the scores as well, and **the avg-max rule lowered none**. **The final model therefore trains with the avg-max rule**. Section 6.6 gathers the evidence.

### 4.4.3 The avg-max rule and its implementation

**The max rule is cheap**. The sigmoid keeps the order of the scores, so **the maximum can run on the logits**, as one masked maximum over the whole batch on the GPU.

**The avg-max rule cannot run on the logits**: the mean of 2 scores is not the sigmoid of the mean of their logits. So it runs on the scores, as the loop of Section 4.4.1, per article and per tree. **Each step of the loop is a mean, which is linear, and the choice of which pairs to average carries no gradient**: a comparison of 2 scores is a yes-or-no decision, and its derivative is 0, so autograd on the tensor version gives the same gradient as the tape, through the means alone. **The forward pass therefore records the pairs it averaged, as a tape, and the backward pass replays the same means in reverse order on the gradient**. **The implementation went through 3 versions**. The first ran the loop on tensors, with a `torch.where` and 2 index writes per step, so autograd tracked each step, fired or not, and the backward pass walked them all; it was far too slow. The second ran the loop in plain code and recorded the means as a matrix $M$ per article, then computed $P = Mw$ under autograd, one linear operation per batch. **The third, in the final model, is the tape**: it records the pairs alone and replays them, with no matrix. The loop runs in float64 on the CPU, compiled with numba, and **takes about as long as the max rule on the GPU**.

The loss needs $\log p$ and $\log(1-p)$. **The loop therefore takes 2 inputs, the score of a topic being present, $\sigma(s)$, and the score of it being absent, $\sigma(-s)$, and applies each mean to both**, so that $\log(1-p)$ needs no subtraction in floating point.

## 4.5 Training settings

**The final model trains the whole encoder** with the settings of Table 4.6. **The optimizer is AdamW with 3 learning rates**: a low one for the encoder, which has to keep its language skill, a middle one for the graph encoder, and a high one for the blocks that start from scratch, the attention, the pool, the head and the projection. **The learning rate warms up over the first steps and then follows a cosine down to a floor**. **The batch is 16 articles, whatever the GPU**. A large GPU holds all 16 in 1 step. **A small one holds the same batch through gradient accumulation**: it runs the 16 articles in several smaller steps, sums their gradients, and updates the weights once. A small script picks the split from the GPU's memory and an estimate of the memory one step needs. **A topic with 1 vote out of 3 enters the loss as a soft value**, Section 3.3. **Such a topic has $y = 0.5$ in the loss instead of $y = 1$**, since 1 vote out of 3 is weak evidence, Table 4.6. On the validation and test data each topic is present or absent, $y = 1$ or $y = 0$. **On the gold subset this soft value and the input format with the article-to-topic attention** of Section 4.3.3 **were the 2 changes that raised the score most**. **A run stops when the validation loss has not improved for a few evaluations, and the checkpoint with the best macro average precision is the one kept**.

Table 4.6: Training settings of the final model.

| Setting | Value | Note |
|---|---|---|
| Trained parameters | all layers of the encoder, all blocks | |
| Optimizer | AdamW, weight decay 0.01 | |
| Learning rate, encoder | $3 \times 10^{-5}$ | |
| Learning rate, graph encoder | $1 \times 10^{-4}$ | |
| Learning rate, other blocks | $1 \times 10^{-3}$ | attention, pool, head, projection |
| Schedule | warm-up 6 %, then cosine to 10 % of the peak | |
| Gradient clipping | 1.0 | |
| Effective batch | 16 articles | micro-batch × accumulation, from the GPU's memory |
| Precision | bf16 autocast | fp32 master weights |
| Input length | 4,096 tokens | |
| Soft value | 0.5 for a topic with 1 vote | 2 votes or more count as 1 |
| Early stop | validation loss, patience 3 evaluations | 2 on the full corpus, 2 evaluations per epoch |
| Checkpoint kept | best macro average precision | |

# Chapter 5

# Experimental setup

**Training ran on rented cloud GPUs**, 1 GPU per run, Table 5.1; a laptop held the code and its tests. **Each run writes its configuration, the commit hash of the code and its metrics to a run store on the Hugging Face Hub**, and an index page sorts the runs by their scores.

Table 5.1: The training environment. Table 4.6 gives the training settings.

| Item | Value |
|---|---|
| Cloud GPUs | NVIDIA T4 and L4 for the frozen baselines; A100, L40S, H100 and H200 for the fine-tunes and HGCLR |
| Precision | bf16 autocast, fp32 master weights |
| Attention kernel | flash attention 2 on Ampere and newer, SDPA otherwise |
| Software | Python, PyTorch 2.4 or later, transformers 5, numba for the avg-max rule |
| Run store | Hugging Face Hub, one folder per run with configuration, commit hash and metrics |

# Chapter 6

# Results

**The runs fall into 4 groups**, Table 6.1, by their training data and their code: the easy data set on the old platform, the first gold set on the old platform, the pruned gold set on the rebuilt pipeline, and the same set with 6 topics folded. **Scores compare within a group and not across groups**, so each table names its group. Sections 6.1 to 6.4 use group 1, the later sections groups 2 to 4. A one-seed row uses the same seed throughout; an ensemble averages the scores of its seeds.

Table 6.1: **The 4 groups of runs.** Scores compare within a group, not across groups. Groups 3 and 4 share their validation split.

| Group | Training data | Code | Dates | Real test set |
|---|---|---|---|---|
| 1 | easy data set | old platform | Aug. to mid-Sept. | none |
| 2 | gold set, first version | old platform | 14–16 Sept. | none in the index |
| 3 | gold set, pruned | rebuilt pipeline | 23–30 Sept. | before trimming |
| 4 | gold set, pruned, 6 topics folded | rebuilt pipeline | from 14 Oct. | after trimming |

## 6.1 Baselines on the easy data set: encoders, frozen and fine-tuned

Table 6.2 gathers the baselines of Section 4.2 on the easy data set, from the run index of Appendix C. **The rows are not all on the same footing**: some have 1 seed and some an ensemble of seeds, and the frozen head changed its learning rate in September. The source comment of the table names the run behind each row.

3 results stand. **EuroBERT beats multilingual-e5** with a frozen encoder; the high recall of the e5 row comes from over-prediction. **The large EuroBERT scores higher than the small one, at several times the time and memory**; so **the 210m is the base of the work**, and the choice between the 2 sizes, trained or frozen, follows the requirements and the production budget. **Training the encoder helps**: the fine-tuned 210m beats the frozen 210m with 1 seed and with the ensemble. The attention pool on the frozen encoder recovers part of that gap, which is why Section 4.3.4 keeps it. The Graphormer label attention of Section 4.2.3 reaches the fine-tuned baseline and does not pass it. LoRA has no row in the index, as Section 4.2.2 says.

## 6.2 Losses and coherence rules on the easy data set

Table 6.3 lists the ASL sweep of Section 4.3.6 on the frozen baseline, 1 seed per run. **A light focusing on the negatives scores the highest macro AP. A higher $\gamma_-$ raises recall and**

Table 6.2: **Group 1: easy data set, old platform.** Baselines, validation split. One seed, or the ensemble of several seeds. Weighted BCE and no coherence rule unless stated.

| Model | Encoder | Seeds | Macro AP | Micro AP | Macro F1 | Macro recall |
|---|---|---|---|---|---|---|
| Frozen + head | EuroBERT 210m | 1 | 0.563 | 0.710 | 0.530 | 0.672 |
| Frozen + head | EuroBERT 210m | ens. | 0.579 | 0.727 | 0.547 | 0.671 |
| Frozen + head | EuroBERT 2.1B | 1 | 0.581 | 0.722 | 0.548 | 0.633 |
| Frozen + head | multilingual-e5 | 1 | 0.547 | 0.697 | 0.435 | 0.848 |
| Frozen + attention pool | EuroBERT 210m | ens. | 0.598 | 0.737 | 0.546 | 0.747 |
| Fine-tune | EuroBERT 210m | 1 | 0.606 | 0.753 | 0.560 | 0.683 |
| Fine-tune | EuroBERT 210m | ens. | 0.627 | 0.773 | 0.584 | 0.689 |
| Fine-tune | EuroBERT 2.1B | 1 | 0.612 | 0.752 | 0.504 | 0.835 |
| Graphormer LA, fine-tune | EuroBERT 210m | ens. | 0.627 | 0.760 | 0.601 | 0.636 |
| Graphormer LA, frozen, max rule + ASL | EuroBERT 210m | ens. | 0.632 | 0.775 | 0.588 | 0.646 |
| Graphormer LA, frozen, max rule + ASL | EuroBERT 2.1B | ens. | 0.624 | 0.767 | 0.581 | 0.634 |

**lowers both F1 and macro AP**. Focusing on the positives alone lowers recall. The adaptive runs, where $\gamma_-$ changes during training, score no better. The weighted BCE row comes from a later run, so it is a rough reference.

Table 6.3: **Group 1: easy data set, old platform.** The ASL sweep on the frozen baseline, 1 seed. The weighted BCE row comes from a later run of the same baseline. Macro AP is the sort key of Chapter 6.

| Loss | $\gamma_-$ | $\gamma_+$ | $m$ | Macro AP | Macro F1 | Macro recall | Micro AP |
|---|---|---|---|---|---|---|---|
| BCE, no weight | 0 | 0 | 0 | 0.571 | 0.507 | 0.449 | 0.726 |
| BCE, capped positive weight | 0 | 0 | 0 | 0.563 | 0.530 | 0.672 | 0.710 |
| ASL | 0 | 0 | 0.05 | 0.575 | 0.502 | 0.450 | 0.724 |
| ASL | 1 | 0 | 0.05 | **0.581** | 0.540 | 0.571 | 0.740 |
| ASL | 2 | 0 | 0.025 | 0.575 | 0.542 | 0.653 | 0.725 |
| ASL | 2 | 0 | 0.05 | 0.575 | 0.538 | 0.673 | 0.726 |
| ASL | 2 | 0 | 0.1 | 0.570 | 0.529 | 0.682 | 0.721 |
| ASL | 3 | 0 | 0.05 | 0.559 | 0.528 | 0.655 | 0.710 |
| ASL | 4 | 0 | 0.05 | 0.550 | 0.522 | 0.649 | 0.706 |
| ASL | 6 | 0 | 0.05 | 0.553 | 0.504 | 0.695 | 0.705 |
| ASL | 6 | 0 | 0.1 | 0.554 | 0.492 | 0.721 | 0.705 |
| ASL | 0 | 1 | 0 | 0.567 | 0.463 | 0.377 | 0.722 |
| ASL, adaptive $\gamma_-$, gap 0.1 | – | 0 | 0.05 | 0.554 | 0.514 | 0.695 | 0.707 |
| ASL, adaptive $\gamma_-$, gap 0.05 | – | 0 | 0.05 | 0.553 | 0.519 | 0.664 | 0.706 |
| ASL, adaptive $\gamma_-$, gap 0.01 | – | 0 | 0.05 | 0.553 | 0.521 | 0.657 | 0.706 |
| ASL, adaptive $\gamma_-$, gap 0.005 | – | 0 | 0.05 | 0.552 | 0.521 | 0.661 | 0.706 |

Table 6.4 compares the coherence rules of Section 4.4 in pairs that differ in the rule alone. **On the easy data set neither rule hurts the model**: with the max rule or the avg-max rule, in the loss or at inference, the scores stay within the noise of 1 seed, and **a rule in the loss keeps the predictions coherent during training**. **The gold training set tells the 2 rules apart**, Section 6.6.

## 6.3 HGCLR and our changes on the easy data set

Table 6.5 gives the HGCLR runs of Section 4.3 on the easy data set, 1 seed each. 2 findings shaped the final model.

**The frozen encoder served to compare the masks.** With the encoder frozen, a run takes minutes, so I used it to **compare ways to build the positive view and to explore**

Table 6.4: **Group 1: easy data set, old platform.** Coherence rules. Each block differs in the rule alone. The last 3 columns give macro F1 with no rule, with the max rule and with the avg-max rule applied to the saved scores; a run trained with a rule in the loss has the same value in all 3.

| Model | Rule in training | Seeds | Macro AP | Macro recall | F1 flat | F1 max | F1 avg-max |
|---|---|---|---|---|---|---|---|
| Frozen, ASL | none | ens. | 0.592 | 0.582 | 0.552 | 0.552 | 0.550 |
| Frozen, ASL | max (C-HMCNN) | ens. | 0.593 | 0.580 | 0.557 | 0.557 | 0.557 |
| Frozen, ASL | none | 1 | 0.581 | 0.571 | 0.540 | 0.540 | 0.540 |
| Frozen, ASL | avg-max | 1 | 0.581 | 0.591 | 0.546 | 0.546 | 0.546 |
| Fine-tune, BCE | none | ens. | 0.627 | 0.689 | 0.584 | 0.583 | 0.586 |
| Fine-tune, BCE | max (C-HMCNN) | ens. | 0.630 | 0.689 | 0.584 | 0.584 | 0.584 |

**how many tokens the mask keeps**. Table 6.5 shows the differences, all from 1 seed:

- **The keep rate moves recall more than ranking.** A mask that keeps 90 % of the tokens, guided or random, raises macro recall by about 9 points and lowers macro F1 by about 3 against no mask, while macro AP drops by less than 0.5 point. Keeping 80 % lowers macro AP by about 1 point.
- **The guided mask edges out the random one.** At the same rate of 90 %, the mask guided by the topic names scores a little higher than the random mask on macro AP and recall.
- **No mask with the contrastive loss scores highest**, a little above the frozen baseline and above no mask without the contrastive loss. The run without the contrastive loss trained for more epochs, so this gap mixes 2 changes.

With a frozen encoder **the contrastive loss reaches the head and not the encoder** whose vectors it should shape, so all these gains stay small. **The encoder has to train**, so all later work fine-tunes it.

**With a trained encoder HGCLR improves the metrics.** The paper's schedule, a positive view at each step and all layers trained, **scored below the plain fine-tune and trained much slower**. 2 changes reversed this: **training the top layers of the encoder alone, and building the positive view on 1 batch out of several**, so that the other batches train like a plain fine-tune and skip the second pass through the encoder. **The best run then beats the plain fine-tune of the same seed by about 1 point of macro AP**.

The 2 blocks of Sections 4.3.3 and 4.3.4 came in with the final model on the gold training set, so they have no HGCLR row here.

## 6.4 Summary on the easy data set

**The easy data set settled the design before the gold training set arrived**: it shaped the final structure of the model and gave a good starting set of settings. **EuroBERT 210m is the base encoder**, trained with a lower learning rate and a warm-up; **the loss is ASL** with a light focusing on the negatives; **a coherence rule removes the violations** at no cost in ranking; **HGCLR improves the metrics** with the top layers trained and a sparse schedule of positive views; and **the attention pool is worth its cost**.

## 6.5 Runs on the gold training set

Table 6.6 gives the runs of the rebuilt pipeline on the full pruned gold set, in groups 3 and 4.

Table 6.5: **Group 1: easy data set, old platform.** HGCLR, 1 seed each, weighted BCE unless stated. Frozen rows: the positive view comes from a mask that keeps the stated share of tokens; a guided mask keeps the tokens closest to the names of the article's topics. Fine-tune rows: the positive view comes from the graph-guided generator of Section 4.3.2. The positive views of a frozen run are encoded once and cached; the 2 rows with no mask differ only in how often the cache is rebuilt, which with no mask changes the random stream alone. The plain fine-tune row repeats Table 6.2.

| Encoder | Positive view | Contrastive loss | Macro AP | Micro AP | Macro F1 | Macro recall |
|---|---|---|---|---|---|---|
| frozen | none, the article itself; cache built once | yes | 0.567 | 0.714 | 0.536 | 0.670 |
| frozen | none, the article itself; cache rebuilt every 8 epochs | yes | 0.565 | 0.713 | 0.537 | 0.653 |
| frozen | guided mask, 90 % | yes | 0.564 | 0.709 | 0.506 | 0.761 |
| frozen | guided mask, 90 %, re-scored by the head | yes | 0.564 | 0.708 | 0.500 | 0.768 |
| frozen | random mask, 90 % | yes | 0.563 | 0.707 | 0.503 | 0.756 |
| frozen | none, the article itself | no | 0.560 | 0.708 | 0.534 | 0.663 |
| frozen | random mask, 80 % | no | 0.548 | 0.694 | 0.509 | 0.709 |
| frozen baseline, no HGCLR | – | – | 0.563 | 0.710 | 0.530 | 0.672 |
| fine-tune, all layers | generator, every step | yes | 0.595 | 0.742 | 0.556 | 0.675 |
| fine-tune, top 2 layers | generator, 1 batch in 10 | yes | 0.597 | 0.737 | 0.550 | 0.697 |
| fine-tune, top 2, adaptive ASL | generator, 1 batch in 10 | yes | 0.604 | 0.755 | 0.455 | 0.858 |
| fine-tune, top 6 layers | generator, 1 batch in 4 | yes | 0.613 | 0.756 | 0.571 | 0.677 |
| fine-tune, top 6 layers | generator, 1 batch in 6 | yes | 0.616 | 0.757 | 0.574 | 0.680 |
| plain fine-tune, no HGCLR | – | – | 0.606 | 0.753 | 0.560 | 0.683 |

- **The HGCLR design beats the base**, group 4. **The base** fine-tunes the same EuroBERT 210m with the article-to-topic attention, the attention pool and the head, on the same data, input length and schedule, and leaves out 5 parts: the graph encoder, the positive view with its contrastive loss, the token FFN, ASL in place of binary cross-entropy, and the coherence rule. With 1 seed the HGCLR design scores about 2 points of macro AP above the base, on the validation data and on the real test set; the gap belongs to these 5 parts together.
- **The ensemble of 3 seeds adds about as much again**, group 4, Section 6.7. It is the final model.
- **The encoder size matters more than the input length**, group 3: the 610m on 1,024 tokens scores above the 210m on 2,048 tokens, on the validation data and on the real test set. **The max rule in the loss hurts**: the 610m trained with it loses about 2 points of macro AP against the 610m trained without a rule. **The avg-max rule hurts less**, Section 6.6, which is why the final model trains with it. The 2 610m runs differ in their soft values and article-to-topic attention as well, so part of the loss may belong to those settings.

Table 6.6: **Groups 3 and 4: pruned gold set, rebuilt pipeline**, full corpus. Compare rows within a group only. One seed unless stated.

| | | | | Validation | | Real test | | |
|---|---|---|---|---|---|---|---|---|
| Model | Encoder | Input | Rule | Macro AP | Micro AP | Macro AP | Micro AP | Violation rate |
| **Group 3**: all topics in training; real test set before trimming, 10,000 articles | | | | | | | | |
| HGCLR design | 210m | 2,048 | avg-max | 0.810 | 0.892 | 0.654 | 0.779 | 0 |
| Earlier HGCLR design | 610m | 1,024 | none | **0.824** | **0.897** | **0.685** | **0.796** | 0.009 |
| HGCLR design | 610m | 1,024 | max | 0.799 | 0.885 | 0.646 | 0.774 | 0 |
| **Group 4**: 6 weak topics folded; real test set after trimming, 9,701 articles | | | | | | | | |
| Base: no graph encoder, positive view, token FFN, ASL or rule | 210m | 2,048 | none | 0.785 | 0.879 | 0.630 | 0.764 | 0.007 |
| HGCLR design, 1 seed, mean of 3 | 210m | 2,048 | avg-max | 0.804 | 0.890 | 0.652 | 0.780 | 0 |
| HGCLR design, ensemble of 3 seeds: final model | 210m | 2,048 | avg-max | **0.823** | **0.903** | **0.683** | **0.803** | 0 |

## 6.6 Choice of the coherence rule

Table 6.7 gathers the evidence for the rule from both run indexes.

- **After training, the max rule lowers macro F1 and the avg-max rule raises it**, in most of the runs trained without a rule, panel A.
- **In the loss, the max rule lowers macro AP on the gold data and the avg-max rule does not**, panel B.

**The final model therefore trains and predicts with the avg-max rule.** Chapter 8 lists what the record leaves open.

Table 6.7: The evidence for the avg-max rule. **Each pair in panel B sits inside one group of Table 6.1.** A: each rule applied after training to the saved scores of the 58 runs of the old platform trained without a rule. B: the rule inside the loss, macro AP on the validation data; a dash marks a pair that was not run. *The 2 runs also differ in the soft values and the article-to-topic attention.

| Group | Runs | No rule | Max | Avg-max |
|---|---|---|---|---|
| **A.** Rule at inference, macro F1 | | | | |
| 1 and 2 | 58 runs, mean | 0.494 | 0.492 | 0.496 |
| | runs above no rule | – | 4 of 58 | 52 of 58 |
| **B.** Rule in the loss, macro AP | | | | |
| 1 | frozen + head, ASL, ensemble | 0.592 | 0.593 | – |
| 1 | frozen + head, ASL, 1 seed | 0.581 | – | 0.581 |
| 2 | frozen + head, BCE, ensemble | 0.717 | 0.702 | – |
| 3, subset | HGCLR, 210m, 1 seed | 0.738 | 0.744 | 0.743 |
| 3 | HGCLR, 610m, 1 seed* | 0.824 | 0.799 | – |

## 6.7 Seeds and the ensemble

Table 6.8 gives the 3 seeds of the final design, group 4.

- **The 3 seeds score almost the same**: the seed changes macro AP by less than 0.5 point.
- **Their ensemble scores above each seed**, by about 2 points of macro AP on the validation data and 3 on the real test set, at 3 times the inference cost.

Table 6.8: The final design with 3 seeds and their ensemble. **Group 4**, full corpus.

| | Validation | | | Real test | |
|---|---|---|---|---|---|
| Seed | Macro AP | Micro AP | Macro F1 | Macro AP | Micro AP |
| 42 | 0.805 | 0.890 | 0.719 | 0.654 | 0.781 |
| 43 | 0.804 | 0.890 | 0.721 | 0.650 | 0.779 |
| 44 | 0.805 | 0.890 | 0.719 | 0.653 | 0.782 |
| ensemble | **0.823** | **0.903** | **0.738** | **0.683** | **0.803** |

# Chapter 7

# Error analysis

**The metrics of this chapter skip 32 topics that the model learns**, Table 7.1. They come from a review of the weakest topics one by one, on the dashboard and in the per-topic tables of the real test set. Each had a reason: the topic is unlearnable, since it overlaps with its neighbours and scores low on the generated data too; or it shifts, fine on generated articles and weak on real press; or the real test set holds too few of its articles to measure it. All 32 are leaves whose parent still catches their articles, so the team ruled that the model keeps learning them and the metrics skip them.

Table 7.1: The 32 topics the model learns and the metrics skip, by the reason found for each.

| Group | Topics | Meaning | Examples |
|---|---|---|---|
| Unlearnable | 11 | weak on the generated validation data as well: the topic overlaps with its neighbours | *record and achievement*, *civil rights*, *business service* |
| Shift | 9 | fine on the generated data, weak on real press | *birthday*, *surveillance*, *freedom of the press* |
| Thin | 12 | under 30 articles in the real test set | *coup d'etat*, *philosophy*, *volunteering* |

## 7.1 The errors by level

Table 7.2 splits the scores of the final model by level, on the validation data and on the real test set. 2 patterns hold. **The deeper the level, the lower the score**: level 1 is almost solved, level 3 is where the model errs. And **each level scores lower on real articles than on generated ones**, with the largest drop at levels 2 and 3. **On the validation data the model predicts more topics per article than the LLMs gave**, so precision is the weak side there.

## 7.2 Errors on the validation data: the model or the LLM topics?

**On the validation data a disagreement between the model and the LLM topics has 3 possible causes**: the model is wrong, the LLM topics are wrong, since they are votes and not truth, or the article sits between 2 similar topics and either answer holds.

**The weakest topics are the ones with the fewest training articles.** Figure 7.1 shows the view of my dashboard per topic for the final model on the validation data: on the left, each topic's AP against its number of training articles; on the right, the topics ranked by AP inside each level. **The topics below the macro AP line are level 2 and 3 topics with a few**

Table 7.2: The final model by level. Macro AP, and precision and recall over all article-topic pairs at the threshold of 0.5.

| | | Validation, generated | | | Test, real | | |
|---|---|---|---|---|---|---|---|
| Level | Topics | Macro AP | Precision | Recall | Macro AP | Precision | Recall |
| 1 | 17 | 0.965 | 0.859 | 0.943 | 0.907 | 0.821 | 0.803 |
| 2 | 106 | 0.848 | 0.731 | 0.889 | 0.711 | 0.689 | 0.742 |
| 3 | 168 | 0.792 | 0.615 | 0.821 | 0.642 | 0.599 | 0.667 |
| all | 291 | 0.823 | 0.725 | 0.884 | 0.683 | 0.705 | 0.743 |
| Topics per article, LLM / model | | 12.8 / 15.6 | | | 11.3 / 11.9 | | |

**hundred articles.** The figure leaves out the 32 topics that the model learns and the metrics skip, as Table 7.2 does. **So the errors sit where the data is thin**, and the question is what kind of error they are.

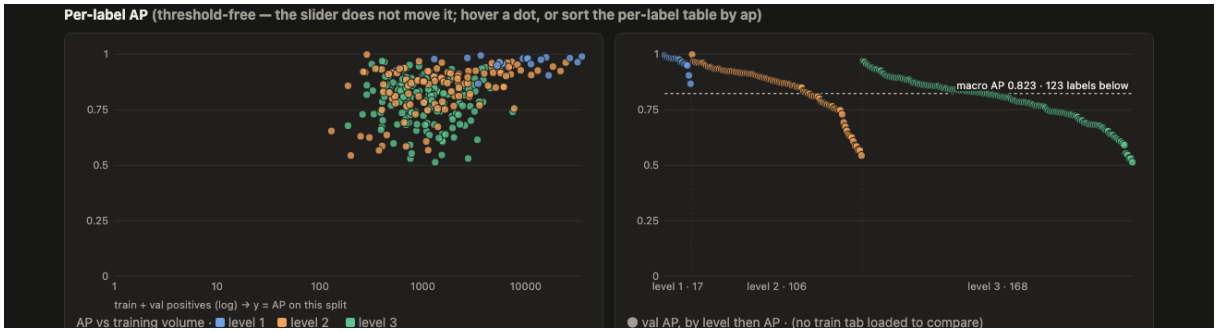



Figure 7.1: The per-topic view of the dashboard, final model, validation data, over the 291 scored topics. Left: AP of each topic against its number of training articles, log scale, 1 dot per topic, coloured by level. Right: the topics ranked by AP inside each level, with the macro AP line.

**To tell the causes apart I read the articles.** 2 scripts pick them: 1 sorts the validation articles by the gap between the model's scores and the LLM topics, the other by the number of wrong level 1 topics. Table 7.3 gives 1 article per cause, and Table 7.4 the full topic list of 1 of them. **Each cause asks for a different fix.** Wrong LLM topics ask for a judge pass on levels 2 and 3, as the pruning of Section 3.3 did for level 1. Missed topics lower the measured precision of the model and ask for the same judge. Model errors ask for more articles on the weak topics, since they are the rare ones. The hard cases stay: a soft value for a topic with 1 vote is the right target for them.

To measure this, I let an LLM judge a random sample of such disagreements, blind to which side proposed each topic. Table 7.5 gives the verdicts. For a topic the model adds, the model is wrong in most cases, so its precision is a real weakness. For a topic the model misses, the LLM topic is wrong in more than 50 % of the cases, so a large part of the measured recall gap is noise in the topics, not in the model. The audit ran on an earlier run of the same design, with a single LLM judge and no human check.

## 7.3 Errors on the real test set: a different mix of topics

An LLM wrote the training and validation articles; the test articles are real French press. The 2 differ less in style than in their mix of topics. Table 7.6 gives the share of articles that carry each level 1 topic. Politics and economy fill the generated articles; lifestyle, weather, environment and the arts are far more common in real press. The model scores lower on the topics that grew:

Table 7.3: 4 causes of a disagreement on the validation data, 1 article each. The LLM topics and the model's topics at level 1 and 2, where they differ.

| Cause | Article | LLM topics vs model | What the text says |
|---|---|---|---|
| The LLM topics are wrong | *Orange Metalic: 3 headliners for a sold-out night* | LLMs: weather, with phenomena, warning, forecast; model: lifestyle, arts, economy | a concert report of 2,500 people; the weather is 1 line on a local alert at the end of the night |
| The LLM topics miss a topic | *A retired woman from Ajaccio knits a 21 km scarf*, Table 7.4 | LLMs: no *senior citizens*; model: 0.96 | the text is about a 72-year-old retired woman |
| The model is wrong | *French stand-up comedians tour the country this autumn* | LLMs: arts and entertainment, theatre; model adds economy, business enterprise, start-up at 0.5 to 0.6 | a tour of shows; the text names the organisers, and the business is not the subject |
| Hard for a human too | the same scarf article | LLMs: *fashion*, 2 votes; model: 0.05 | a knitted scarf as a record; fashion fits the object, not the story |

Table 7.4: A validation article where the model and the LLM topics disagree. *Une retraitée d'Ajaccio tricote une écharpe de 21 kilomètres, chantier textile inédit en Corse*: a 72-year-old retired woman finishes a scarf 21 km long; the body stops at a paywall after 2 sentences. Votes: LLMs out of 3 that gave the topic; 0 means no LLM did.

| Topic | Votes | Score |
|---|---|---|
| human interest | 3 | 1.00 |
| human interest > record and achievement | 3 | 0.66 |
| lifestyle and leisure | 3 | 0.49 |
| lifestyle and leisure > leisure | 2 | 0.31 |
| arts, culture, entertainment and media > arts and entertainment > fashion | 2 | 0.05 |
| society > demographic group > senior citizens | 0 | 0.94 |
| society > communities | 0 | 0.71 |

Table 7.5: Blind LLM judge on validation disagreements, earlier run of the same design. One judge, no human check.

| Disagreement | Model wrong | LLM topics wrong |
|---|---|---|
| model adds a topic the LLMs did not give | 88 % | 6 % |
| model misses a topic the LLMs gave | 26 % | 56 % |

it misses part of lifestyle and weather, and it gives *human interest* to 50 % of the real articles while the LLMs give it to 33 %. Selecting the checkpoint on generated validation data hides this shift.

Table 7.6: Share of articles that carry a level 1 topic, in the generated validation data and in the real test set, and the model's results on the real test set at the threshold of 0.5. Topics with the largest change.

| | LLM topics | | Model, real test set | | |
|---|---|---|---|---|---|
| Level 1 topic | Generated | Real | Predicted | Precision | Recall |
| politics and government | 60 % | 38 % | 35 % | 0.92 | 0.84 |
| economy, business and finance | 46 % | 36 % | 34 % | 0.86 | 0.82 |
| crime, law and justice | 27 % | 17 % | 15 % | 0.88 | 0.78 |
| lifestyle and leisure | 6 % | 28 % | 22 % | 0.89 | 0.69 |
| environment | 9 % | 22 % | 19 % | 0.92 | 0.79 |
| weather | 2 % | 13 % | 9 % | 0.97 | 0.65 |
| arts, culture, entertainment and media | 16 % | 24 % | 25 % | 0.78 | 0.82 |
| human interest | 29 % | 33 % | 52 % | 0.59 | 0.92 |

The worst real articles, ranked by a script on the number of level 1 errors, are in many cases not articles at all. Table 7.7 shows the third worst: an author page of *Libération* that lists the latest headlines of its photo service. The LLMs tagged it with the topics of the listed stories; the model reads a page about the Middle East and predicts diplomacy. Index pages, daily digests, video pages and copies of the same text fill the top of this ranking. On an earlier run, removing them all raised F1 by less than 0.5 point, so they explain a small part of the gap.

Table 7.7: A real test page with many errors. *Service Photo : ses derniers articles publiés*, an author page of *Libération* that lists headlines: the war in the Middle East, the PSG victory, a heat wave. Missed: a topic the LLMs gave and the model scored below 0.5. Added: the reverse.

| Topic | Error | Score |
|---|---|---|
| human interest | missed | 0.42 |
| health | missed | 0.41 |
| environment | missed | 0.35 |
| sport | missed | 0.27 |
| weather | missed | 0.19 |
| science and technology | added | 0.75 |
| politics and government > international relations > diplomacy | added | 0.94 |

## 7.4 Lessons for the next model

3 lessons follow. **Work on precision, not recall**: most topics the model adds are wrong, while many of the topics it misses are noise in the LLM votes, so a lower threshold would chase the noise. **Select and calibrate on real articles**: the generated validation data has another topic mix, so a checkpoint or a threshold chosen on it does not carry over to real press. On an earlier run, a threshold per topic tuned on real articles raised F1 by about a point. **Clean the real input**: a filter should remove index pages and digests before the model sees them; the gain in the scores is small, but the product should not tag such pages at all.

# Chapter 8

# Limits

The head is a three-layer MLP from the first frozen baseline on. I did not test it against the single linear layer of HGCLR, nor against other depths, so the depth of the head is an untested choice.

The avg-max rule went into the loss of the final model on 2 results: the max rule in the loss lowered the scores against no rule, and the avg-max rule at inference lowered none. The pair that closes the argument, the avg-max rule in the loss against no rule, exists on a subset of the gold corpus alone, with 1 seed, Table 6.7; on the full corpus it is missing. Each pair of that table has 1 seed or 1 ensemble. The 2 610m runs differ in their soft values and article-to-topic attention as well, so part of their gap may belong to those settings.

The avg-max rule in the loss has one open case. A logit below about $-744$ gives a score of 0 in float64; a 0 takes part in no mean, so its $\log p$ is minus infinity and the loss of that batch is not a number. A guard in the training loop skips such a batch, which happened once in the final runs; a clamp on the logits would remove the case.

The easy data set folded its rare topics into their parents and the model dropped the topics below a minimum share of the training rows. HGCLR gains most on rare topics, in its paper and in the per-topic scores of my best run, so the easy data set left out the topics where its gain should be largest, and no run measures HGCLR on them.

# Chapter 9

# Side work: relevance rerankers for thematic search


## 9.1 Problem and data

## 9.2 zerank cross-encoders run locally

## 9.3 GLiNER2 as a zero-shot classifier

## 9.4 Findings

# Chapter 10

# Conclusion

The internship built a model that tags French press articles with the topics of the IPTC tree, trained on generated articles and tested on real ones. 3 parts of the work are new.

- **HGCLR rebuilt on EuroBERT, with 3 changes.** Attention pooling replaces the first token that EuroBERT lacks, an article-to-topic attention block brings the topic tree to the token states at inference, and the asymmetric loss replaces binary cross-entropy. On the gold data this design beats the same encoder without these parts, and an ensemble of 3 seeds is the final model.
- **The avg-max rule.** It keeps the predicted scores coherent along the tree, as the max rule of C-HMCNN does, and moves a parent and its child toward each other instead of copying the child. The rule raises the scores after training where the max rule lowers them, and it trains inside the loss with a tape that records the averaged pairs and replays them backward.
- **A run platform and an error dashboard.** Each run keeps its commit, its configuration and its metrics in one index, so the choices of the project compare on the same footing, and the dashboard traces an error to a topic, an article or a label.

I learned more from this project than from any course so far.

- **Reading a paper is not enough; rebuilding it is.** HGCLR made sense on paper and fell apart on the first run: the positive views were noise while the encoder was frozen, and the generator made each epoch slow. Reproducing the paper's result before changing anything was the step that made the later changes measurable.
- **Measure before concluding.** Profiling found the time bottleneck in the positive-view generator, not in the encoder. The first avg-max implementation ran under autograd and was too slow to use; writing its backward pass by hand made it cheap. A difference of a few hundredths of macro AP is large in this task, so a claim of no difference needs the numbers next to it.
- **The data matters more than the model.** The gap between generated and real articles, the noise in the LLM topics and the rare topics set the scores more than any block of the model. The gold set with its votes, and the soft value for a topic with 1 vote, gave larger gains than most changes of the architecture.
- **Record each run and plan for small GPUs.** Warm-up with a cosine schedule stopped the collapse of the early runs, several seeds told a real gain from noise, gradient accumulation kept the batch the same on any GPU, and the run index let me come back to a result weeks later and trust it.

# Bibliography


[1] Nicolas Boizard et al. Eurobert: Scaling multilingual encoders for european languages, 2025. arXiv preprint; TODO confirm authors and id.

[2] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning (ICML)*, 2020. TODO confirm; NT-Xent as cited by HGCLR.

[3] Eleonora Giunchiglia and Thomas Lukasiewicz. Coherent hierarchical multi-label classification networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] International Press Telecommunications Council. Iptc media topics newscodes. `https://iptc.org/standards/media-topics/`, 2026.

[5] Tal Ridnik, Emanuel Ben-Baruch, Nadav Zamir, Asaf Noy, Itamar Friedman, Matan Protter, and Lihi Zelnik-Manor. Asymmetric loss for multi-label classification. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[6] torchmil developers. torchmil: Attentionpool. `https://torchmil.readthedocs.io/en/latest/api/nn/attention/attention_pool/`, 2025. TODO confirm authors and year.

[7] Zihan Wang, Peiyi Wang, Lianzhe Huang, Xin Sun, and Houfeng Wang. Incorporating hierarchy into text encoder: a contrastive learning approach for hierarchical text classification. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. exploration/docs/2022.acl-long.491.pdf.

[8] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[9] Jie Zhou, Chunping Ma, Dingkun Long, Guangwei Xu, Ning Ding, Haoyu Zhang, Pengjun Xie, and Gongshen Liu. Hierarchy-aware global model for hierarchical text classification. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. exploration/docs/2020.acl-main.104.pdf.

# Appendix A

# Full architecture graph

# Appendix B

# Configuration file and flag semantics

# Appendix C

# Run index

# Appendix D

# LLM labelling prompts

# Appendix E

# Experiment journal excerpts